Programming Assignment II: Summary

CS683: Advanced Computer Architecture, Autumn 2026
Computer Science and Engineering
Indian Institute of Technology, Bombay

Team Member	Roll No
Tejeswar	24b0918
Bhavesh Ramakrishnan Karthik	24b0945
Sushant Padha	24b1057
Rhythm Kedia	24b1071

Contents

Introduction **3**

Implementation files . . . 3

Methodology . . . 3

Task 1: Stride Pattern Prefetching **4**

Task 1A: IP-stride prefetcher . . . 4

Task 1B: Design . . . 4

Storage overhead . . . 5

Accuracy throttling . . . 5

Results . . . 6

Design choices . . . 8

Task 2A: Instruction-Conscious L2 Replacement Policy (I-RRIP) **10**

Interface change . . . 10

Design: insertion, promotion, eviction . . . 10

Results . . . 12

Task 2B: Dynamic I-RRIP **14**

Leader sets and decision logic 14

Sensitivity study 15

Results 19

Task 3: Incorporating SPP with Dynamic I-RRIP **21**

Prefetch-aware Dynamic I-RRIP 21

Rationale 21

Design choices 22

Results 24

Introduction

Report from team flipflops.

Implementation files

Our code is on the `dev` branch of the repository.

```
prefetcher/spp.l1d_pref             Task 1B: Stride Pattern Prefetcher
replacement/irrip.l2c_repl          Task 2A: I-RRIP
replacement/dyn_irrip.l2c_repl      Tasks 2B and 3: Dynamic I-RRIP
```

Methodology

- Every provided trace is simulated with 25M warmup and 25M simulation instructions.
- The baseline has no L1D prefetcher and LRU replacement at L2C. Speedup is IPC divided by the baseline's IPC on the same trace.

Task 1

Stride Pattern Prefetching

Task 1A: IP-stride prefetcher

We went through and understood the IP Stride prefetcher implementation provided in the GitHub repository. It holds 24 fully-associative LRU IP Trackers with a prefetch degree of 3 (limited within a page boundary). Only the last 8 bits of IP are used in the trackers. Once a particular stride happens two times in a row, it starts prefetching data.

Task 1B: Design

The prefetcher (`prefetcher/spp.l1d_pref`) sits at the L1D and works on the cache-line strides of each load IP. It never prefetches outside the 4 KB page of the access that triggers it. It has three tables:

1. an **IP table** (16 sets $\times$ 6 ways, LRU) that holds each tracked IP's uniform stride and stride signature,
2. a **pattern table** (512 entries, direct mapped) from a signature to the stride that followed it,
3. a **recent-prefetch filter** (256 entries, direct mapped) that drops repeat requests and measures accuracy.

IP table: An IP gets an entry on its first L1D miss. Once tracked, it trains on every load. Each entry keeps the low 16 bits of the IP's last line address, its last stride with a 2-bit confidence that helps to predict the next line on uniform stride mode, a 9-bit stride signature that has information of the past 3 stride length at this IP, and a 2-bit chooser a saturated counter to make sure switches are not too often. A stride larger than 63 lines always leaves the page, so it resets the entry's stride state.

Two Dynamically Chosen Predicors (Per IP): For each new stride s:

- *Uniform*: remember the last stride and predict that it repeats. Its confidence goes up when it repeats and down otherwise. The stored stride is replaced only at confidence 0, so one stray stride does not reset a stable one.
- *Pattern*: predict the stride based on the stride history. Storing the last few strides of every IP, and a map from each history to its next stride, would be too large. Instead the history is hashed into a 9-bit signature, $sig \leftarrow (sig_old \ll 3) \oplus s$, with s as 7-bit two's complement. Each new stride moves the older ones 3 bits up, so the signature covers the last 3 strides: the newest in 7 bits, the one before in 6, the one before that in 3. 2-3 different singature were tested with differing shift and size but no observable change was seen.

A 2-bit chooser per IP starts at "uniform" and moves one step toward the predictor that was right whenever exactly one of them was. The uniform prediction is used when it is confident (confidence

≥ 2) and the chooser is in its upper half, or when the pattern table has no confident prediction. Otherwise the pattern prediction is used. So uniform strides have priority, and an IP falls back to the pattern table only after the pattern table has been right where the uniform stride was wrong.

Pattern table: The 9-bit signature indexes the 512 entries directly. They are shared by all IPs. Each holds a 7-bit stride and a 2-bit confidence. This is the next-stride predictor based purely on the history independent of the IP, reasoning being that storing a pattern table for each IP is too expensive in terms of space.

Issuing: The chosen predictor is walked ahead *degree* steps: $L + s, L + 2s, \ldots$ for a uniform stride. For a pattern, each predicted stride is shifted into a copy of the signature to look up the next one. The walk stops at the first line outside the page, or at a pattern-table entry without confidence. A prefetch fills the L1D unless half of the L1D MSHRs are busy. In that case it fills only the L2 Cache. This make sure that the prefetch doesnt bottleneck the demand misses.

Recent-prefetch filter (RPF): The RPF holds the lines requested recently, indexed by the low 8 bits of the line address with a 10-bit tag. Once a Hit is found the entry is removed so that multi-hit line dont inflate the accuracy. It also records whether each request has filled the L1D, which the accuracy checker uses (see Subsection Accuracy throttling).

Storage overhead

Structure	Entries	Bits per entry	Bits
IP table valid 1, IP tag 12, last line (low bits) 16, stride 7, stride confidence 2, signature 9, chooser 2, LRU 3	96	52	4,992
Pattern table: stride 7, confidence 2	512	9	4,608
Recent-prefetch filter: valid 1, line tag 10, filled 1	256	12	3,072
Throttle: issued 9, useful 9, degree 4	1	22	22
Total			12,694 bits = 1,586.75 bytes

Widths: a stride is at most ± 63 lines, since a larger one always leaves the page, so it takes 7 bits. An IP is identified by 16 hashed bits: 4 pick the set and the other 12 are the tag. LRU among 6 ways needs 3 bits. The epoch counters count to 256, hence 9 bits. The degree goes up to 8, hence 4 bits.

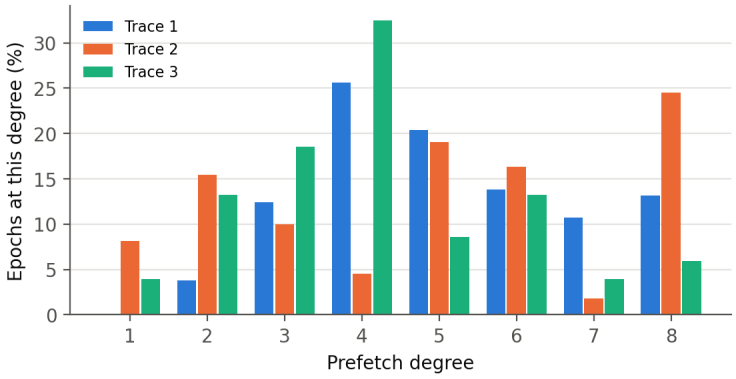
Accuracy throttling

Accuracy checker: The RPF records whether each requested line has filled the L1D.

- A prefetch is counted as issued once it fills the L1D.
- It is useful if a demand load reaches the line while the line is still in the RPF.
- A demand that misses on a requested line that has not filled yet is a late (or L2-only) prefetch. The prediction was right, so it counts as both issued and useful.
- A demand that hits on a requested line that never filled means the line was already cached. That prefetch was redundant and is not counted.

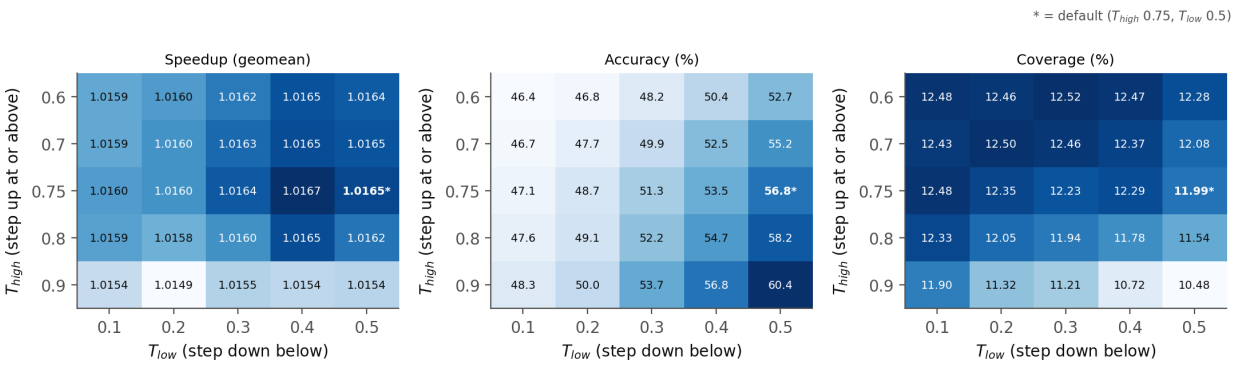
A prefetch that fills only the L2 is counted only if a demand reaches it, so an unused one is not seen.

Degree control: The degree is **not** fixed. It is chosen at run time. Every epoch of 256 counted prefetches, accuracy $a = \textit{useful}/\textit{issued}$ is compared with two thresholds. The degree goes up by one if $a \geq 0.75$, down by one if $a < 0.50$, and otherwise stays. It ranges from 1 to 8 and starts at 4. The degree the throttle settles on differs by trace:



Trace2, where SPP is most accurate, spends the most epochs at the cap of 8. Traces 1 and 3 spend most at degree 4.

Why these thresholds: $T_{\text{low}} = 0.5$ is the break-even point: below it, most prefetches are useless. $T_{\text{high}} = 0.75$ leaves a gap so the degree does not oscillate. Sweep (default starred):

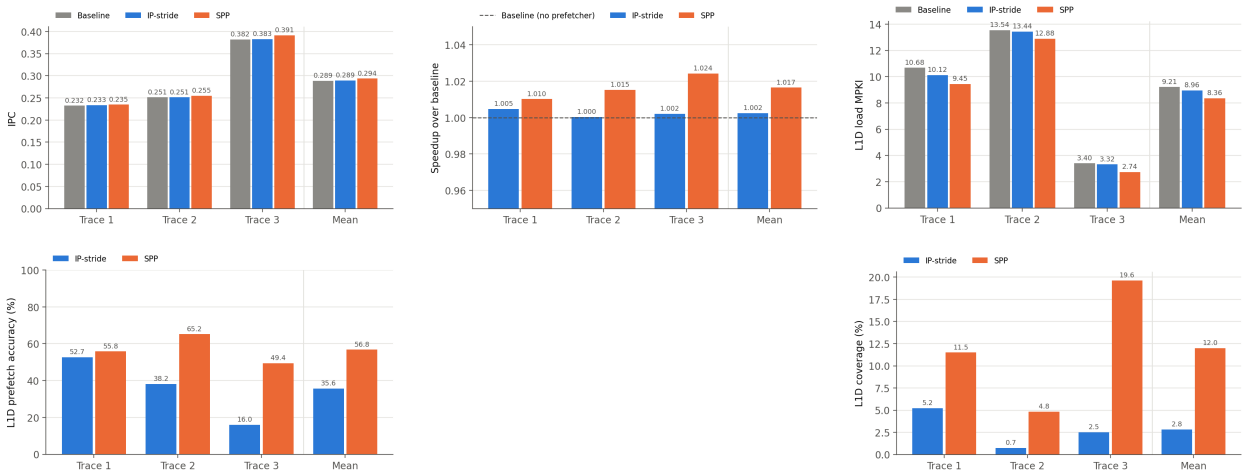


Speedup is flat (1.0149–1.0167); the default is within 0.0002 of the best and, of those, issues the fewest useless prefetches.

Results

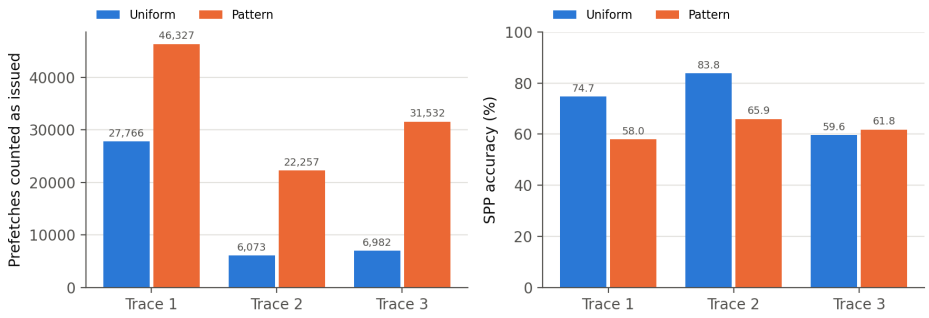
25M warmup and 25M simulated instructions per trace. The baseline has no L1D prefetcher and LRU at the L2C. Accuracy is ChampSim's: useful prefetches / prefetches issued to the L2. Mean: geometric for speedup, arithmetic otherwise.

Trace	Prefetcher	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)
Trace1	none (baseline)	0.2323	1.0000	10.68	–	–
	IP-stride	0.2335	1.0048	10.12	52.7	5.2
	SPP	0.2347	1.0102	9.45	55.8	11.5
Trace2	none (baseline)	0.2512	1.0000	13.54	–	–
	IP-stride	0.2513	1.0004	13.44	38.2	0.7
	SPP	0.2550	1.0153	12.88	65.2	4.8
Trace3	none (baseline)	0.3821	1.0000	3.40	–	–
	IP-stride	0.3829	1.0021	3.32	16.0	2.5
	SPP	0.3913	1.0242	2.74	49.4	19.6
Mean	IP-stride	0.2892	1.0024	8.96	35.6	2.8
	SPP	0.2937	1.0165	8.36	56.8	12.0



SPP is faster than IP-stride on every trace, with higher accuracy and coverage on every trace.

Uniform vs pattern: Both predictors are used on every trace. The pattern table issues most of SPP's prefetches: 63%, 79% and 82% on Traces 1–3. Its accuracy is lower than the uniform predictor's on Traces 1 and 2, and about the same on Trace3:

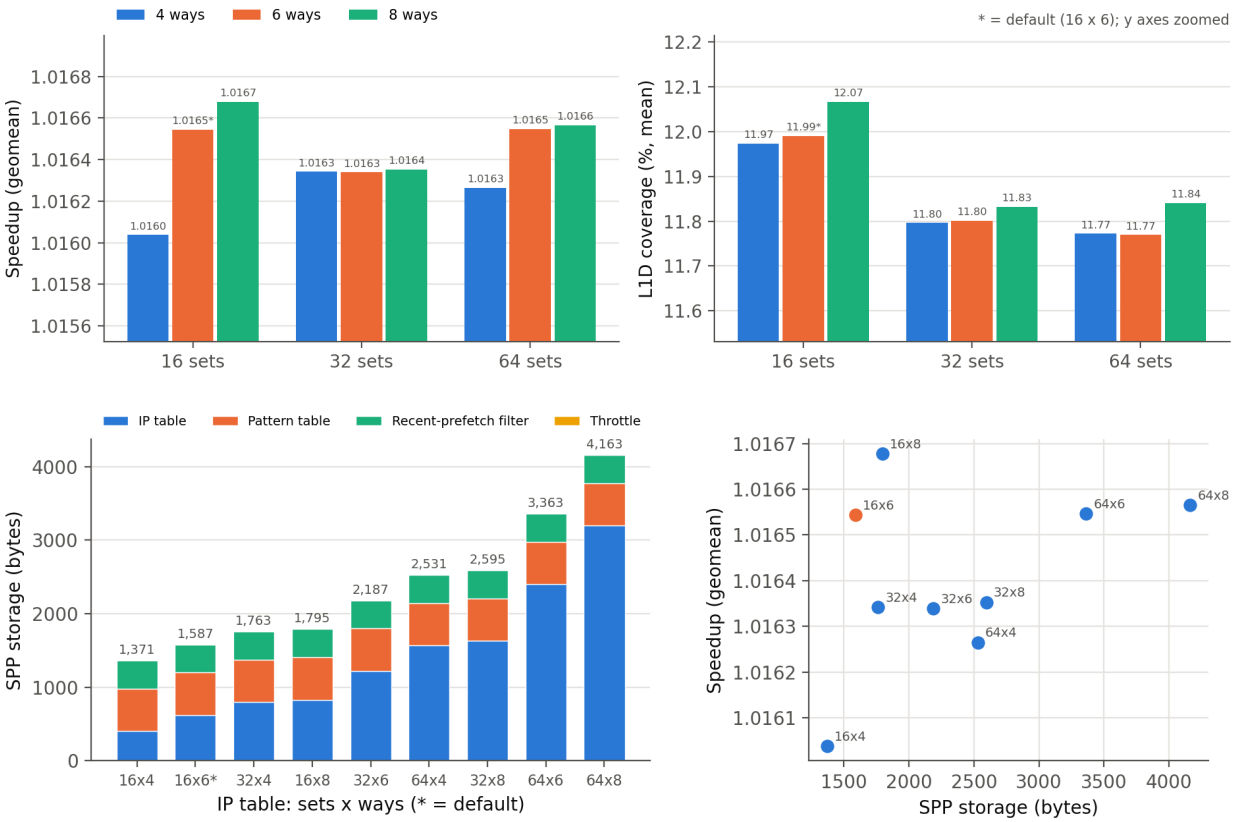


Why the speedup is smaller than the coverage. SPP removes 9% of L1D load misses (mean MPKI 8.36 against 9.21) but gains 1.65% IPC, because of a higher amat obsevered at LLC.

Design choices

IP table size. We swept the IP table's sets and ways, with SPP otherwise at its defaults:

Sets × ways	Entries	Speedup	Coverage (%)	SPP storage (bytes)
16 × 4	64	1.0160	11.97	1,370.75
16 × 6	**96**	**1.0165**	**11.99**	**1,586.75**
32 × 4	128	1.0163	11.80	1,762.75
16 × 8	128	1.0167	12.07	1,794.75
32 × 6	192	1.0163	11.80	2,186.75
64 × 4	256	1.0163	11.77	2,530.75
32 × 8	256	1.0164	11.83	2,594.75
64 × 6	384	1.0165	11.77	3,362.75
64 × 8	512	1.0166	11.84	4,162.75

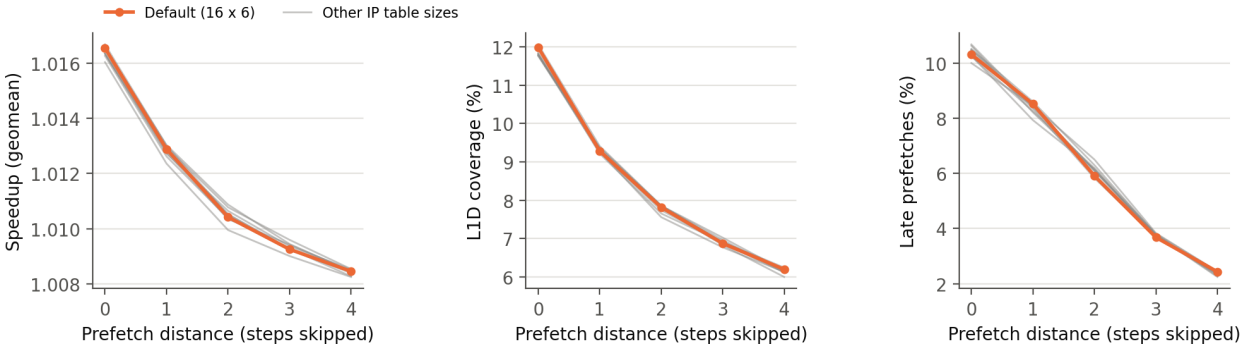


All nine sizes are within 0.0007 of each other in speedup and 0.3 points in coverage, and a larger table does not help steadily. The IP table is the only structure whose size changes, from 30% to 77% of SPP's storage. We use 16 × 6: it is within 0.0002 of the best speedup, at 38% of the storage of the largest table.

Prefetch distance: To make prefetches less late, we tried a constant distance d: the walk goes $degree + d$ steps ahead but issues only the last $degree$ of them, skipping the d nearest lines. Lateness

is the share of correct prefetches that arrive late (the L1D's late-prefetch counter over useful plus late). Means over the three traces, default IP table size:

Distance d	Speedup	Coverage (%)	Accuracy (%)	Late (%)
0	**1.0165**	**12.0**	**56.8**	**10.3**
1	1.0129	9.3	57.1	8.5
2	1.0104	7.8	55.7	5.9
3	1.0093	6.9	54.9	3.7
4	1.0085	6.2	53.2	2.4



A distance does cut lateness, but coverage halves (12.0% to 6.2% at $d = 4$), and speedup drops with every step, for all nine IP table sizes. Without a distance only 10.3% of correct prefetches are late, so there was little to gain and also a larger loss in not prefetching the useful lines that weren't last, a potential Improvement would be learn the distance as function of IP but no implemented due to time constrains. We decided to keep a default constant $d = 0$.

Task 2A

Instruction-Conscious L2 Replacement Policy (I-RRIP)

Interface change

We added a last parameter, `uint8_t isInstruction`, to `update_replacement_state`: in the member-function pointer type in `inc/cache.h`, in every cache level's definition, and at every call site. Each call in `src/cache.cc` passes the `instruction` field of the packet it handles: the MSHR entry on a fill from the lower level, the write queue entry on a writeback that misses, and the read, write or prefetch queue entry on a hit. The BTB calls in `src/ooo_cpu.cc` pass 0. The flag is also set on fills of page-table lines for instruction translations, so I-RRIP protects those lines too; the L2C instruction load MPKI does not count them, because it counts only load requests. I-RRIP stores the flag of each line when the line is filled, because `find_victim` does not receive it.

Design: insertion, promotion, eviction

An L1I miss stalls fetch stage itself. A data miss is often overlapped with independent work in the window. An L2 hit is therefore worth more for an instruction line than for a data line, and I-RRIP gives up some data hits to keep instruction lines. It starts from SRRIP with 2-bit RRPVs (maximum RRPV 3), which inserts every line at RRPV 2, promotes every hit to RRPV 0, and evicts the first line at RRPV 3, aging all lines until one reaches 3.

	SRRIP	I-RRIP (final, level 8)
Insertion	every line at 2	instruction lines at 2 (unchanged); data lines at 3
Promotion	every hit to 0	instruction hit to 0 (unchanged); data hit: RRPV $--$
Eviction	first line at 3; else age all lines	first *data* line at 3, else any line at 3; else age: data lines every round, instruction lines every 4th round

Table 1: Replacement rules of SRRIP and the final I-RRIP.

Eviction rule. Instruction lines age once per 4 aging rounds of data lines. Effectively, in a set that holds at least one data line, the victim search ends before any instruction line is aged. An instruction line is then evicted only if it already has RRPV 3, which it can reach only by aging while its set holds no data line.

Experiments we ran. `irrip.l2c_repl` implements eight variants, which we call levels; the default is level 8, the final policy. Each level changes one rule of a previous one:

- Level 1: data-first eviction, instruction lines aged every 2nd round, every line inserted at 2, instruction hit to 0, data hit RRPV $--$.
- Level 2: level 1, but instruction lines inserted at 0.

- Level 3: level 1, plus a cap of 6 instruction lines per 8-way set: when a set holds 6 or more, the instruction line with the highest RRPV is evicted.
- Level 4: level 2, but data hits promoted to 1. Level 5: level 1, but data hits promoted to 1.
- Level 6: level 1, but instruction lines aged every 4th round.
- Level 7: level 1, but data lines inserted at 3.
- Level 8: levels 6 and 7 together.

		Load MPKI			MPKI ratio to SRRIP		
Policy	Speedup	L2C	Instr.	Data	L2C	Instr.	Data
SRRIP	1.0004	15.81	9.85	5.96	1.000	1.000	1.000
level 1	1.0182	13.55	6.35	7.20	0.887	0.656	1.173
level 2	1.0089	14.47	7.08	7.39	0.934	0.719	1.199
level 3	1.0329	14.99	8.20	6.79	0.979	0.881	1.125
level 4	1.0084	14.50	7.12	7.38	0.936	0.725	1.197
level 5	1.0178	13.56	6.37	7.19	0.888	0.660	1.172
level 6	1.0194	13.85	6.15	7.70	0.904	0.627	1.240
level 7	1.0216	13.73	5.81	7.92	0.901	0.597	1.277
level 8 (I-RRIP)	1.0216	13.91	5.76	8.14	0.911	0.590	1.306

Table 2: Every level, averaged over the three traces. Speedup is over LRU (geometric mean). MPKI columns are arithmetic means; ratio columns are each MPKI relative to SRRIP (geometric mean of the per-trace ratios). Percentages in the text compare the MPKI columns.

Insertion.

- *Instruction lines: not modified (RRPV 2).* Inserting them at 0 (level 2 vs. level 1) raises mean instruction MPKI by 11% and mean data MPKI by 3%, and lowers the speedup from 1.0182 to 1.0089. At RRPV 2, only instruction lines that are reused at the L2 get full protection. At RRPV 0, every instruction fill gets full protection, including lines that are not reused before they are evicted, and **data-first eviction** keeps them for a long time.
- *Data lines: modified (RRPV 3).* Level 7 vs. level 1 lowers instruction MPKI by 8.6% and raises data MPKI by 10%, and the speedup rises from 1.0182 to 1.0216. A data line inserted at 3 is a victim candidate ahead of every instruction line unless it is hit first, so data that is not reused soon cannot push instruction lines out. A hit moves it to RRPV 2; it is still evicted before any instruction line once it ages back to 3.

Promotion.

- *Instruction hits: not modified (RRPV 0).* A hit is the only evidence of reuse, and RRPV 0 gives a reused instruction line the most protection which helps drastically based on the experiments.
- *Data hits: modified (RRPV − 1).* The data promotion rule barely matters (due to strong instruction protection in all subpolicies). We varied this but did not see considerable changes.

Eviction.

- *Modified: data first, instruction lines aged more slowly.* Aging instruction lines every 4th round instead of every 2nd (level 6 vs. level 1) lowers instruction MPKI by 3.1% and raises data MPKI by 6.9%, for a speedup of 1.0194 instead of 1.0182. This is evidence that keeping instructions in the L2C is more important than keeping data.
- On top of data insertion at 3, slower aging adds very little (level 8 vs. level 7): instruction MPKI falls by 0.8%, data MPKI rises by 2.8%, and the speedup is the same (1.0216). **We keep level 8 because it has the lowest instruction MPKI, which is the goal of this task.**

The instruction cap – odd results (level 3). Level 3 has the highest speedup (1.0329), but its **instruction MPKI is 29% above level** 1's, so it does not meet the goal of this task, and we did not choose it. The cap lowers LLC load misses (Table 3): e.g. on Trace1, from 301.9k with level 1 to 267.4k, while L2C load misses rise from 447.4k to 464.5k. The reason we chose 6 comes from a sweep where 6 showed highest speedup.

Trace	Policy	L2C load misses	LLC load hits	LLC load misses
Trace1	LRU	442.4	135.5	306.9
	level 1	447.4	145.5	301.9
	level 3	464.5	197.1	267.4
	level 8	465.0	166.2	298.8

Table 3: L2C load misses and LLC load hits and misses, in thousands, over the 25M simulated instructions.

Results

Trace	Policy	IPC	Speedup	L2C	L2C instr.	L2C data
Trace1	LRU (baseline)	0.2323	1.0000	17.70	10.34	7.36
	SRRIP	0.2322	0.9995	17.64	10.56	7.08
	I-RRIP	0.2339	1.0068	18.60	8.98	9.62
Trace2	LRU (baseline)	0.2512	1.0000	27.36	18.85	8.50
	SRRIP	0.2518	1.0026	25.75	17.78	7.96
	I-RRIP	0.2639	1.0509	19.25	7.63	11.62
Trace3	LRU (baseline)	0.3821	1.0000	3.95	1.12	2.83
	SRRIP	0.3817	0.9991	4.04	1.20	2.84
	I-RRIP	0.3850	1.0076	3.87	0.67	3.19
Mean	LRU (baseline)	0.2885	1.0000	16.34	10.11	6.23
	SRRIP	0.2886	1.0004	15.81	9.85	5.96
	I-RRIP	0.2943	1.0216	13.91	5.76	8.14

Table 4: IPC, speedup over LRU, and L2C load MPKI (total, instruction, data). L2C load MPKI is the sum of instruction and data load MPKI. Mean: geometric for speedup, arithmetic otherwise.

		Load MPKI			MPKI ratio to LRU		
Policy	Speedup	L2C	Instr.	Data	L2C	Instr.	Data
LRU (baseline)	1.0000	16.34	10.11	6.23	1.000	1.000	1.000
SRRIP	1.0004	15.81	9.85	5.96	0.986	1.009	0.967
I-RRIP	1.0216	13.91	5.76	8.14	0.898	0.595	1.263

Table 5: Means over the three traces (geometric for speedup, arithmetic for MPKI), and each MPKI relative to LRU (geometric mean of the per-trace ratios).

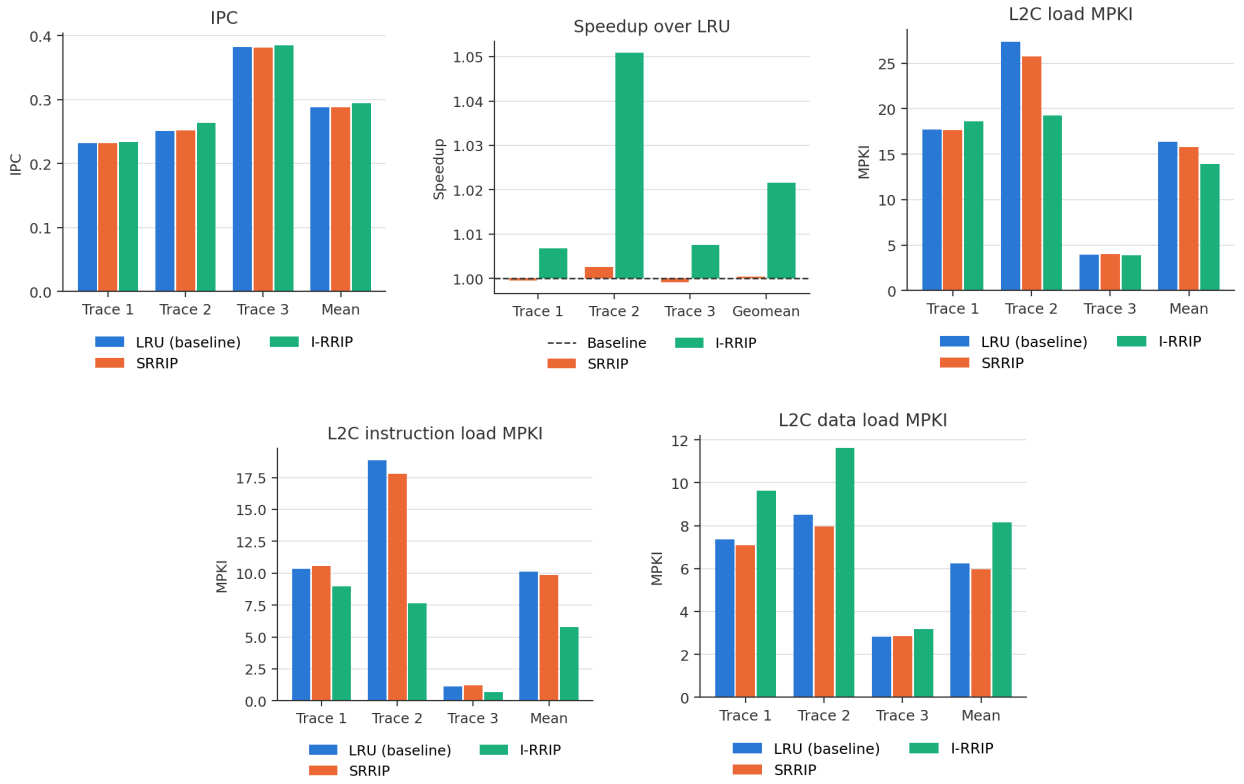


Figure 1: IPC, speedup over LRU (dashed line), and L2C load MPKI per trace.

I-RRIP lowers L2C instruction load MPKI on every trace: by 13% on Trace1, 60% on Trace2 and 40% on Trace3. Data load MPKI rises on every trace, by 31%, 37% and 13%. SRRIP alone is within 0.3% of LRU on every trace, so the gain comes from the instruction priority, not from RRIP.

Instruction misses cost more than data misses. On Trace1, I-RRIP has 5% *more* L2C load misses than LRU (18.60 vs. 17.70 MPKI), yet its IPC is 0.7% higher: it trades 1.35 MPKI of instruction misses for 2.26 MPKI of data misses. This is consistent with the standard argument.

Task 2B

Dynamic I-RRIP

Leader sets and decision logic

Mechanism. A few L2C sets, the *leaders*, always run SRRIP. In the leaders we count instruction accesses, instruction misses, all accesses and all misses, over demand traffic only (writebacks and prefetches are not counted). The other sets, the *followers*, run I-RRIP (from Task 2A) or SRRIP, as decided from the leaders' counts. Every line records on fill whether it is an instruction line, in every set and in both modes, so a follower that switches mode keeps valid state; both modes use the same 2-bit RRPV scale.

Decision. We use the formula from the problem statement,

$$r = \frac{\text{instr. accesses} + \text{instr. misses}}{\text{total accesses} + \text{total misses}},$$

and the followers run I-RRIP when $r > T$, SRRIP otherwise.

Epochs. The decision is made once per *epoch*, every E demand accesses to the leaders. After each decision the four counters are halved. This has the effect of giving greater weight to recent epochs so that counters don't converge to the whole-run average and stop the policy from ever switching. **We finally use 1024 epochs.**

Why the leaders run SRRIP. If the leaders ran I-RRIP, the ratio we mentioned would be skewed anyway. It would require a more complex interpretation of the threshold (perhaps simple threshold will not even suffice). Reason: I-RRIP removes instruction misses, which lowers r and can switch the followers to SRRIP, which raises instruction misses again. With SRRIP leaders, r measures the workload in an easy to interpret manner so we can set a direct threshold.

Leader placement choice. There are N leaders among the 1024 sets, and rule is: **set s is a leader when its low k index bits equal the next k bits** (it is called "complement placement"), where $1024/2^k = N$ (for $N = 32$: `(s & 31) == ((s >> 5) & 31)`). Leaders then sit at different offsets in the index, so an access pattern that maps only to some set offsets cannot dominate the sample. We also compared this placement with two others (low index bits all 0, and the first N sets): the speedups differ by less than 0.1% for $T \leq 0.25$ (Table 9).

How many leaders. Each leader set gives up on some I-RRIP gain, since it always runs SRRIP. With 8 or 16 leaders the policy makes no SRRIP decision after warmup, so their loss against static I-RRIP (0.02% and 0.04%, Table 7) is the cost of the leader sets alone. More leaders would only help in getting a less noisy estimate for r; but in our runs we saw this never occured. More leaders also make each epoch shorter in instructions (an epoch counts leader accesses), so the decision follows shorter dips in r and switches more often, which costs speedup: 32 and 64 leaders switch to SRRIP on Trace2 and Trace3 and lose 0.44% and 0.52% (see the sensitivity study below). **We use 8 leaders finally**.

Final parameters (the code defaults): 8 leaders, complement placement, $E = 1024$ leader accesses, $T = 0.15$, hysteresis $h = 0.05$ (see the extension below).

Sensitivity study

We did "pilot" runs over wider ranges (8–256 leaders, epochs of 128–8192, T from 0 to 1) to find some reasonable ranges to do intensive testing: T below 0.15 changes nothing, and from $T = 0.7$ on every trace runs SRRIP throughout. The threshold, leader, epoch and placement sweeps use no hysteresis ($h = 0$); hysteresis is studied seperately later. All results are averages over the three traces; the ratio columns are each MPKI relative to static I-RRIP (geometric mean of the per-trace ratios).

Threshold T (8 leaders, $E = 512$):

			Load MPKI			MPKI ratio to I-RRIP		
T	IPC	Speedup	L2C	Instr.	Data	L2C	Instr.	Data
SRRIP	0.2886	1.0004	15.81	9.85	5.96	1.098	1.694	0.766
I-RRIP	0.2943	1.0216	13.91	5.76	8.14	1.000	1.000	1.000
0	0.2942	1.0214	13.91	5.79	8.12	1.000	1.005	0.997
0.15	0.2942	1.0213	13.90	5.79	8.11	1.000	1.005	0.996
0.25	0.2940	1.0208	13.91	5.81	8.10	1.001	1.029	0.992
0.4	0.2925	1.0159	14.15	6.21	7.94	1.021	1.161	0.972
0.5	0.2916	1.0123	14.09	6.78	7.31	1.020	1.258	0.905

Table 6: Threshold sweep. Speedup over LRU (geometric mean); MPKI is the arithmetic mean.

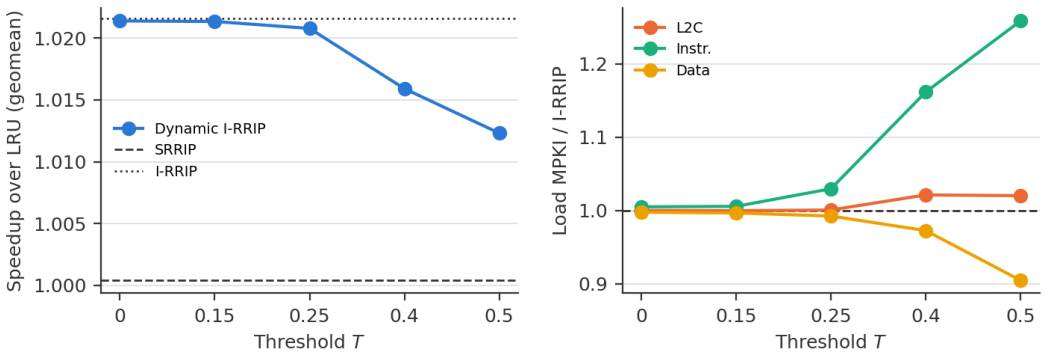


Figure 2: Threshold sweep: speedup over LRU, and each MPKI relative to I-RRIP.

Speedup falls as T rises: from 1.0214 at $T = 0$ to 1.0123 at $T = 0.5$. A higher T means more epochs decide to run SRRIP; e.g. at $T = 0.5$, Trace1 runs SRRIP in 10 of its 14 decisions. SRRIP epochs raise instruction MPKI more than they lower data MPKI, and as Task 2A showed, instruction misses cost more. The same holds at every leader count and epoch length we ran.

Leader count ($T = 0.15$, $E = 1024$): Table 7, discussed above.

			Load MPKI			MPKI ratio to I-RRIP		
Leaders	IPC	Speedup	L2C	Instr.	Data	L2C	Instr.	Data
SRRIP	0.2886	1.0004	15.81	9.85	5.96	1.098	1.694	0.766
I-RRIP	0.2943	1.0216	13.91	5.76	8.14	1.000	1.000	1.000
8	0.2942	1.0214	13.91	5.79	8.12	1.000	1.005	0.997
16	0.2942	1.0211	13.92	5.83	8.09	1.001	1.013	0.995
32	0.2929	1.0171	14.02	6.07	7.94	1.008	1.092	0.975
64	0.2926	1.0163	14.05	6.16	7.89	1.014	1.132	0.970

Table 7: Leader count sweep ($T = 0.15$, $E = 1024$). 8 and 16 leaders make no SRRIP decision after warmup, so their loss against I-RRIP is the cost of the leader sets running SRRIP; 32 and 64 leaders switch to SRRIP on Trace2 and Trace3 (more analysis later).

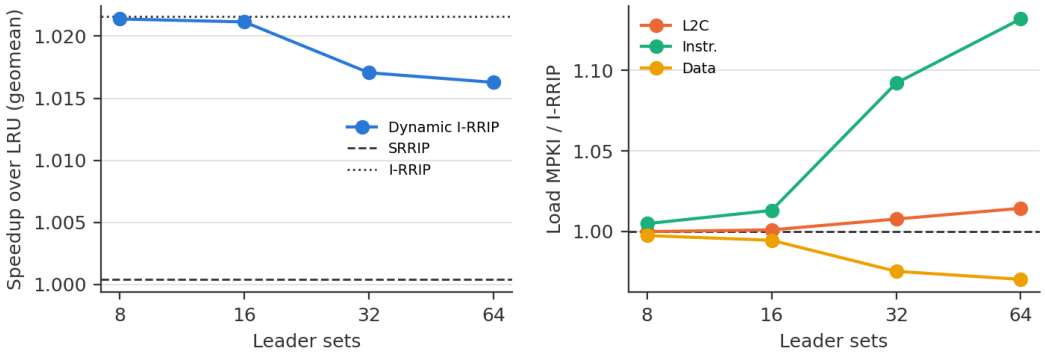


Figure 3: Leader count sweep: speedup over LRU, and each MPKI relative to I-RRIP.

Epoch length (32 leaders, $T = 0.15$):

			Load MPKI			MPKI ratio to I-RRIP		
Epoch	IPC	Speedup	L2C	Instr.	Data	L2C	Instr.	Data
SRRIP	0.2886	1.0004	15.81	9.85	5.96	1.098	1.694	0.766
I-RRIP	0.2943	1.0216	13.91	5.76	8.14	1.000	1.000	1.000
512	0.2927	1.0166	13.99	6.04	7.94	1.010	1.112	0.975
1024	0.2929	1.0171	14.02	6.07	7.94	1.008	1.092	0.975
2048	0.2941	1.0208	13.94	5.88	8.07	1.002	1.022	0.992

Table 8: Epoch length sweep, in leader accesses.

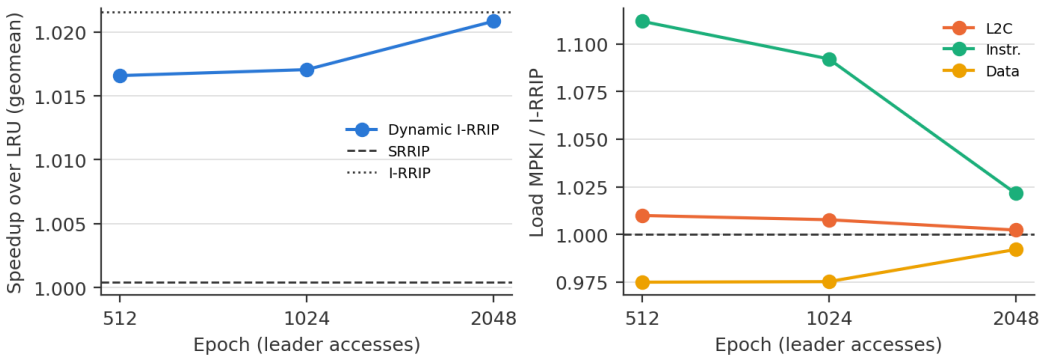


Figure 4: Epoch length sweep: speedup over LRU, and each MPKI relative to I-RRIP.

Longer epochs are better, but only because they decide less often: each decision averages r over more instructions, so short dips in r no longer reach SRRIP. At $E = 2048$ no trace makes an SRRIP decision after warmup.

Placement (32 leaders, $E = 1024$):

T	complement select	low bits = 0	first sets
0	1.0209	1.0206	1.0207
0.15	1.0171	1.0173	1.0165
0.25	1.0163	1.0161	1.0160
0.4	1.0144	1.0126	1.0133
0.5	1.0124	1.0108	1.0121

Table 9: Speedup over LRU for the three leader placements rules. This justifies our choice of "complement select" for default.

Why switching does not help. The ratio does vary within a trace: e.g. with 32 leaders and $E = 1024$, r on Trace2 ranges from 0.07 to 0.70 across epochs. Figure 5 shows r at every decision on Trace1, with a short epoch ($E = 128$, 8 leaders) so the whole run gives about 95 decisions. After warmup r moves between 0.36 and 0.60 in slow waves, not single outliers. With $T = 0.4$ the policy picks SRRIP in 2 of 52 decisions after warmup, at the bottom of two of these dips, and Trace1's speedup over LRU drops from 1.0073 ($T = 0.15$, no SRRIP decision) to 1.0054; static I-RRIP gets 1.0068. So two SRRIP decisions out of 52 cost 0.14–0.19% of speedup (against static I-RRIP or against $T = 0.15$), while running SRRIP for the whole of Trace1 costs 0.72%. But no configuration we ran beats static I-RRIP on average, and the more decisions choose SRRIP, the lower the speedup. The ratio measures how much of the L2 traffic is instructions (with misses weighted as twice as much as hits). It turned out that the epochs where r is low are not epochs where SRRIP wins, since switching to SRRIP in them lowered speedup.

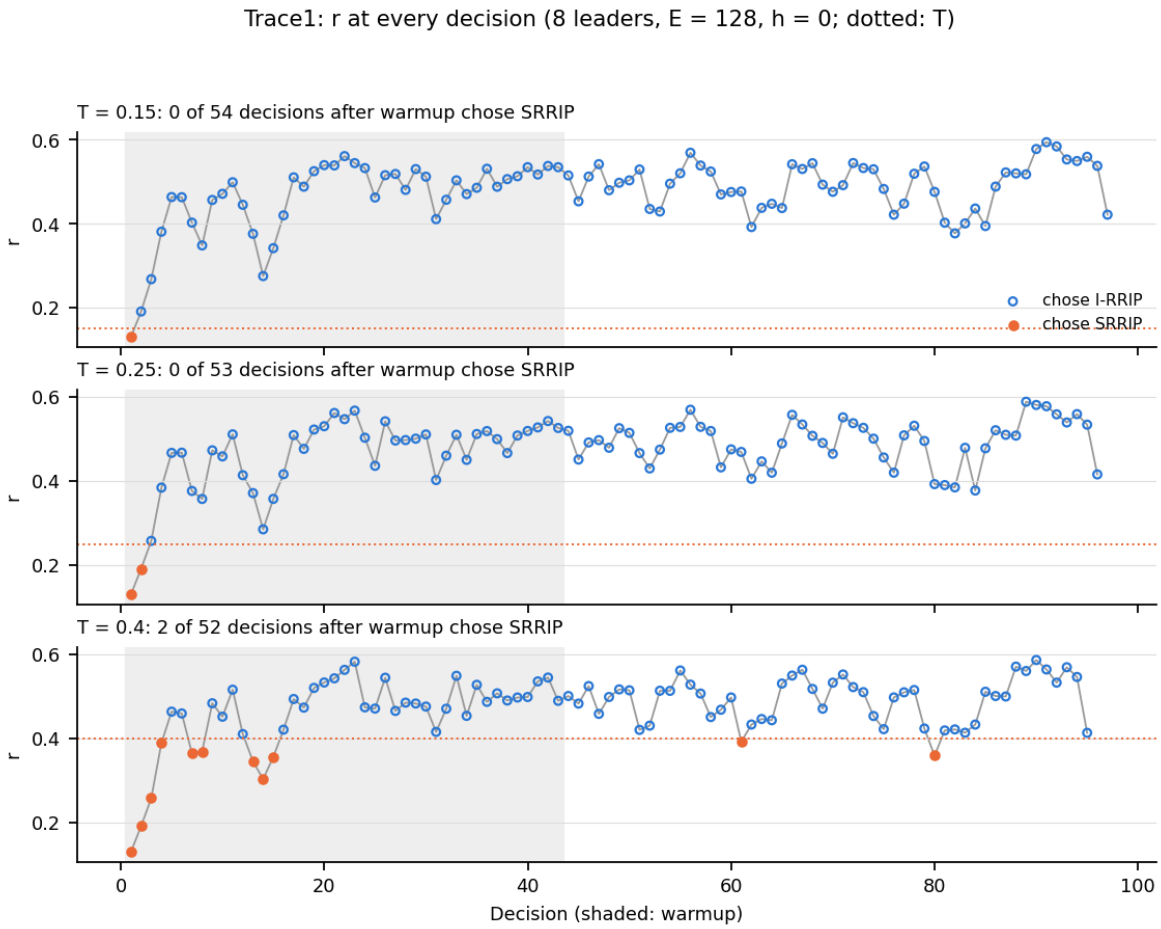


Figure 5: Trace1: r at every decision for $T = 0.15$, 0.25 and 0.4 (8 leaders, $E = 128$, $h = 0$). Hollow markers chose I-RRIP, filled markers chose SRRIP; dotted line: T; shaded: decisions made during warmup

Extension: hysteresis. The problem statement allows extending the formula. With a single threshold, a small dip of r just below T flips the mode, and those switches cost speedup. We added a band of $\pm h$: switch to I-RRIP only when $r > T + h$, and to SRRIP only when $r \leq T - h$.

			Load MPKI			MPKI ratio to I-RRIP		
h	IPC	Speedup	L2C	Instr.	Data	L2C	Instr.	Data
SRRIP	0.2886	1.0004	15.81	9.85	5.96	1.098	1.694	0.766
I-RRIP	0.2943	1.0216	13.91	5.76	8.14	1.000	1.000	1.000
0	0.2929	1.0171	14.02	6.07	7.94	1.008	1.092	0.975
0.05	0.2939	1.0200	13.91	5.93	7.98	1.000	1.028	0.984
0.1	0.2941	1.0208	13.94	5.88	8.07	1.002	1.022	0.992

Table 10: Hysteresis sweep (32 leaders, $E = 1024$, $T = 0.15$).

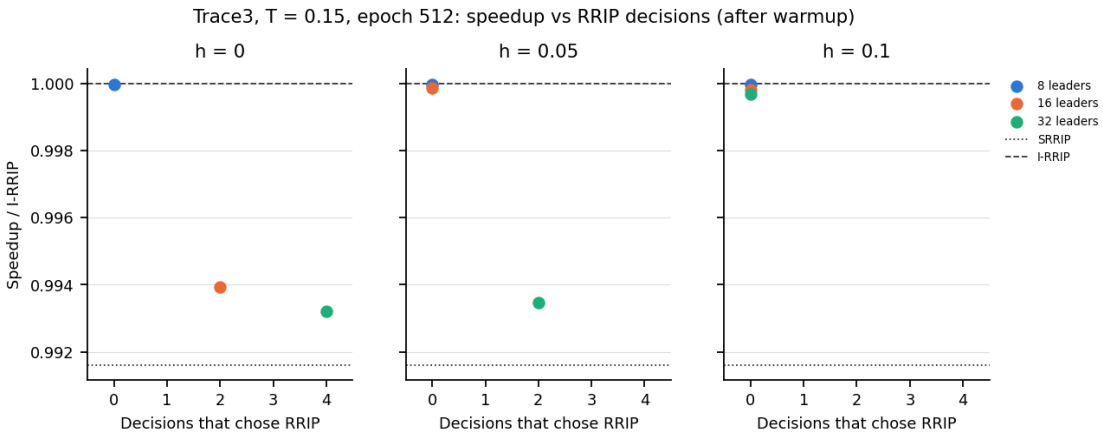


Figure 6: Trace3, $T = 0.15$, $E = 512$: speedup relative to I-RRIP against the number of decisions after warmup that chose SRRIP, one panel per h, one point per leader count.

Hysteresis raises speedup, but that's obvious from above graph: it removes SRRIP decisions. Since every SRRIP decision the ratio makes costs speedup on these traces, hysteresis can at best make the policy never switch. It did not improve on static I-RRIP. **We still keep $h = 0.05$ in the final configuration.** There it gives results identical to $h = 0$, since the policy makes no SRRIP decision either way. Among the configurations that switch, it raised speedup by 0.2–0.3% in two (16 leaders at $E = 512$, 32 leaders at $E = 1024$) and changed it by at most 0.03% in the other three. **It guards against noisy switching on workloads whose r stays near T.**

Results

Trace	Policy	IPC	Speedup	L2C	L2C instr.	L2C data
Trace1	LRU (baseline)	0.2323	1.0000	17.70	10.34	7.36
	SRRIP	0.2322	0.9995	17.64	10.56	7.08
	I-RRIP	0.2339	1.0068	18.60	8.98	9.62
	Dynamic I-RRIP	0.2339	1.0067	18.59	8.99	9.60
Trace2	LRU (baseline)	0.2512	1.0000	27.36	18.85	8.50
	SRRIP	0.2518	1.0026	25.75	17.78	7.96
	I-RRIP	0.2639	1.0509	19.25	7.63	11.62
	Dynamic I-RRIP	0.2638	1.0504	19.27	7.69	11.58
Trace3	LRU (baseline)	0.3821	1.0000	3.95	1.12	2.83
	SRRIP	0.3817	0.9991	4.04	1.20	2.84
	I-RRIP	0.3850	1.0076	3.87	0.67	3.19
	Dynamic I-RRIP	0.3850	1.0076	3.86	0.68	3.19
Mean	LRU (baseline)	0.2885	1.0000	16.34	10.11	6.23
	SRRIP	0.2886	1.0004	15.81	9.85	5.96
	I-RRIP	0.2943	1.0216	13.91	5.76	8.14
	Dynamic I-RRIP	0.2942	1.0214	13.91	5.79	8.12

Table 11: IPC, speedup over LRU, and L2C load MPKI (total, instruction, data) of Dynamic I-RRIP with the final parameters. Mean: geometric for speedup, arithmetic otherwise.

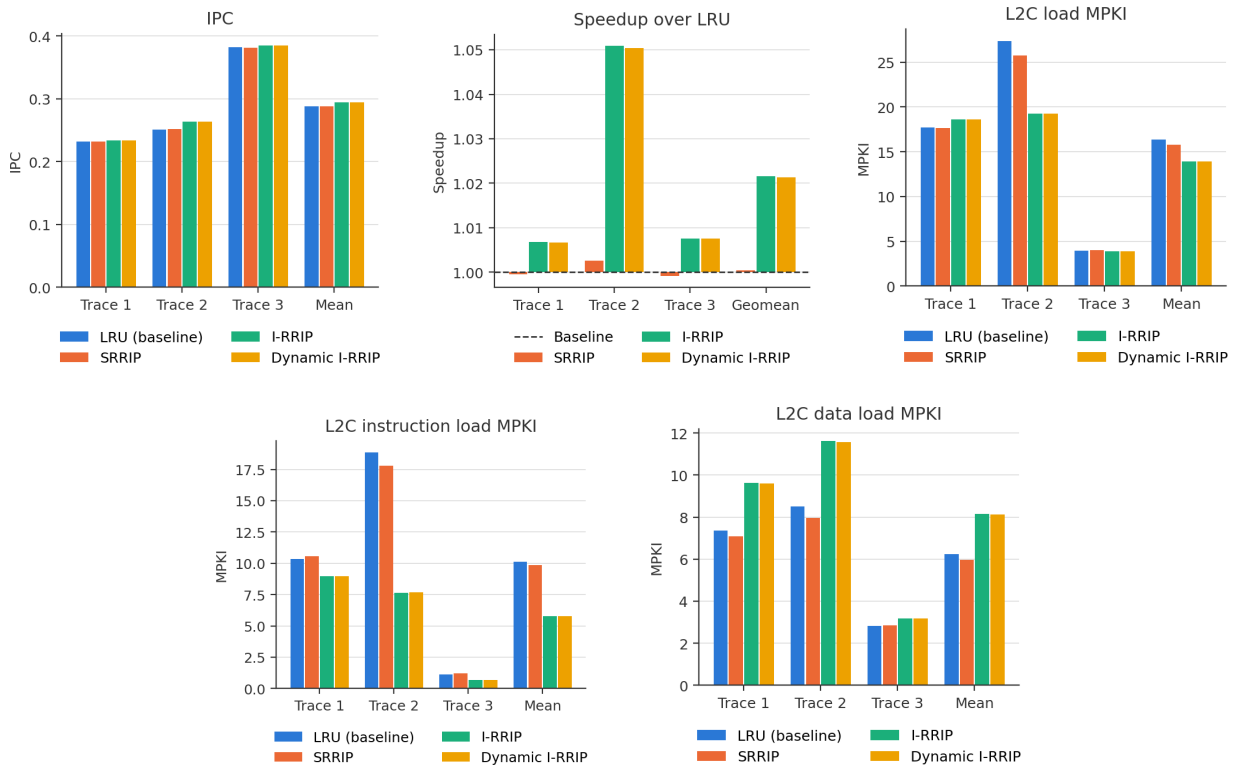


Figure 7: IPC, speedup over LRU (dashed line), and L2C load MPKI per trace.

Dynamic I-RRIP reaches a speedup of 1.0214 over LRU, against 1.0216 for static I-RRIP, and the same L2C load MPKI (13.91). With the final parameters it makes no SRRIP decision after warmup on any trace: at every decision r is at least 0.46 on Trace1, 0.28 on Trace2 and 0.39 on Trace3, well above $T = 0.15$. So on these traces it behaves as static I-RRIP minus the cost of the 8 leader sets (0.02%). It decides slowly: with 8 leaders, an epoch of 1024 leader accesses gives 7, 13 and 2 decisions over the 25M simulated instructions of Trace1, Trace2 and Trace3. Hysteresis makes switching even rarer still.

Task 3

Incorporating SPP with Dynamic I-RRIP

Prefetch-aware Dynamic I-RRIP

Dynamic I-RRIP (Task 2B) keeps 8 *leader* sets on plain RRIP. Every 256 leader accesses (prefetches and writebacks not counted) it computes the instruction share of their accesses and misses, and runs all other (*follower*) sets as I-RRIP if the share is above 0.15, else as RRIP. Task 3 adds one rule to the follower sets: **a prefetch never raises a line's priority.**

- *Prefetch fill:* inserted at RRPV 3, the first to be evicted (a demand fill goes in at RRPV 2 in RRIP mode).
- *Prefetch hit:* no promotion (Task 2B promoted it to RRPV 0 in RRIP mode, by one step in I-RRIP mode).

Demand and writeback hits promote as before. Leader sets stay plain RRIP, so the mode decision is still made on an unmodified policy. The rule reads only the packet type and needs no storage.

Rationale

The L2C copy of a prefetched line is idle. An SPP prefetch that misses the L1D fills the L2C and the L1D together. While the L1D holds the line, demands hit there, so the L2C copy can only be hit after the L1D evicts it. A prefetch that hits the L2C is filled into the L1D again, so the L2C copy is idle again. (When half the L1D MSHRs are busy, SPP fills only the L2C; these are 1.9–4.5% of the prefetches that reach the L2C.)

Under plain RRIP, prefetched lines are the least reused. We record for every L2C line what filled it and whether a demand hit it before eviction (Table 12). With every set on RRIP, 3.9–7.6% of prefetched lines get a demand hit, against 10.1–18.6% of demand data and 12.9–17.8% of instruction lines.

	Prefetch fill			Demand data			Instruction		
L2C policy	T1	T2	T3	T1	T2	T3	T1	T2	T3
RRIP	7.6	3.9	5.7	18.6	15.9	10.1	12.9	14.6	17.8
RRIP + rule	4.0	2.0	2.8	18.9	15.9	10.3	13.0	14.6	18.4
I-RRIP	3.3	2.2	2.5	3.4	4.7	2.3	14.7	20.1	24.0
I-RRIP run, leader sets	8.4	4.9	5.2	17.4	16.3	13.6	13.1	13.2	19.0
Dyn. I-RRIP + rule	3.2	2.3	2.3	3.4	5.2	2.3	14.7	21.5	26.3

Table 12: Percentage of evicted L2C lines that got at least one demand hit before eviction, by what filled them, with SPP at the L1D (25M simulated instructions). RRIP, I-RRIP: follower sets pinned to that mode. Rule: the prefetch rule. Leader sets always run RRIP; they evict only 100–300 prefetched lines per trace, so that row's prefetch column is rough. T1–T3: Traces 1–3.

Deadness measured under I-RRIP is inflated. I-RRIP inserts every data line at RRPV 3 and evicts data before instructions, so data lines die young and evict each other, while instruction lines stay. Demand data reuse drops from 10.1–18.6% under RRIP to 2.3–4.7% under I-RRIP, yet the leader sets of the same I-RRIP runs, which stay on RRIP, still reuse 13.6–17.4% of their data: the lines are not dead by nature, I-RRIP kills them. Table 13 shows the shift: I-RRIP evicts more data lines and up to 2.3× fewer instruction lines, and instruction reuse rises to 14.7–24.0%. Prefetched lines are inflated the same way (2.2–3.3% reused under I-RRIP). So the case for the rule rests on the RRIP numbers above, not on the I-RRIP ones.

Victim filled by	Policy	T1	T2	T3
Demand data	RRIP	238	263	86
	I-RRIP	307	378	96
Instruction	RRIP	265	432	33
	I-RRIP	225	192	18
All	RRIP	618	789	171
	I-RRIP	776	793	218

Table 13: L2C evictions (thousands, all sets) by what filled the victim, follower sets pinned to RRIP or I-RRIP, SPP on, prefetch rule off.

Where the rule acts. In I-RRIP mode a prefetch fill is already a data fill at RRPV 3, so only the hit rule changes anything. In RRIP mode both parts are new. The followers spend 86–100% of the epochs in I-RRIP mode, so the rule can only have a small effect on the dynamic policy.

Design choices

We ran each change with all follower sets pinned to one mode, to separate it from the mode decision (Table 14).

- *Prefetch fill at RRPV 3:* no change in I-RRIP mode; in RRIP mode it lowers L2C load MPKI by 0.7% and instruction load MPKI by 0.7%. Kept.

- *No promotion on prefetch hits:* lowers L2C load MPKI by 0.2% in I-RRIP mode; with the fill rule, by 0.7% (instruction 0.8%) in RRIP mode. Kept.
- *Evict unused prefetched lines first* among lines at RRPV 3: about the same MPKI reduction, but it needs a prefetch bit per line. Dropped.

Across all combinations the speedup moves by at most 0.05%: the misses saved are under 1% of the L2C misses. We kept the two changes that need no storage and lower misses in both modes.

					Load MPKI		Ratio to first row	
Mode	Hit	Insert	Evict	Speedup	L2C	Instr.	L2C	Instr.
I-RRIP	0	–	0	1.0393	12.99	5.78	1.000	1.000
	1	–	0	1.0397	12.99	5.78	0.998	1.000
	0	–	1	1.0397	12.99	5.78	0.998	1.000
	1	–	1	1.0398	12.98	5.78	0.998	0.999
RRIP	0	0	0	1.0177	14.90	9.75	1.000	1.000
	1	0	0	1.0176	14.87	9.72	0.998	0.998
	0	1	0	1.0180	14.83	9.69	0.993	0.993
	1	1	0	1.0179	14.83	9.68	0.993	0.992
	0	0	1	1.0180	14.83	9.68	0.994	0.992
	1	0	1	1.0178	14.82	9.66	0.994	0.990
	0	1	1	1.0179	14.82	9.68	0.994	0.991
	1	1	1	1.0178	14.81	9.67	0.993	0.990

Table 14: SPP + Dynamic I-RRIP with follower sets pinned to I-RRIP or RRIP mode. 1 = change on. Hit: no promotion on a prefetch hit. Insert: prefetch fill at RRPV 3 (data is inserted at RRPV 3 in I-RRIP mode anyway). Evict: evict an unused prefetched line first among lines at RRPV 3. Speedup: geometric mean over traces; MPKI: arithmetic mean; ratios: geometric mean of per-trace ratios to the first row of each mode. Final policy: Hit = 1, Insert = 1, Evict = 0.

Results

Trace	Config	IPC	Speedup	Accuracy (%)	Coverage (%)
Trace1	SPP	0.2347	1.0099	55.9	11.4
	Dyn. I-RRIP	0.2339	1.0067	–	–
	SPP + Dyn. (unaware)	0.2367	1.0187	55.7	11.4
	SPP + Dyn. (aware)	0.2368	1.0190	55.6	11.2
Trace2	SPP	0.2549	1.0150	64.4	4.7
	Dyn. I-RRIP	0.2629	1.0468	–	–
	SPP + Dyn. (unaware)	0.2664	1.0606	63.4	4.7
	SPP + Dyn. (aware)	0.2665	1.0611	63.6	4.7
Trace3	SPP	0.3917	1.0252	49.7	20.4
	Dyn. I-RRIP	0.3837	1.0044	–	–
	SPP + Dyn. (unaware)	0.3949	1.0337	50.1	20.4
	SPP + Dyn. (aware)	0.3946	1.0329	50.0	20.3
Mean	SPP	0.2938	1.0167	56.7	12.2
	Dyn. I-RRIP	0.2935	1.0191	–	–
	SPP + Dyn. (unaware)	0.2993	1.0375	56.4	12.1
	SPP + Dyn. (aware)	0.2993	1.0375	56.4	12.1

Table 15: IPC, speedup over the baseline (no L1D prefetcher, LRU at L2C), and L1D prefetch accuracy and coverage. Accuracy: useful prefetches over prefetches sent to the L2C; coverage: fraction of baseline L1D load misses removed. Aware: the final policy. Unaware: the same policy with the rule off. Mean: geometric for speedup, arithmetic otherwise.

Trace	Config	L1D	L2C	L2C instr.	L2C data
Trace1	SPP	9.47	16.74	10.39	6.35
	Dyn. I-RRIP	10.68	18.59	8.99	9.60
	SPP + Dyn. (unaware)	9.47	17.34	8.99	8.35
	SPP + Dyn. (aware)	9.48	17.33	8.98	8.35
Trace2	SPP	12.90	26.80	18.90	7.90
	Dyn. I-RRIP	13.55	19.23	7.94	11.30
	SPP + Dyn. (unaware)	12.91	18.70	8.03	10.67
	SPP + Dyn. (aware)	12.91	18.66	7.97	10.68
Trace3	SPP	2.71	3.12	1.15	1.97
	Dyn. I-RRIP	3.40	3.90	0.77	3.13
	SPP + Dyn. (unaware)	2.71	2.98	0.68	2.31
	SPP + Dyn. (aware)	2.71	2.99	0.68	2.32
Mean	SPP	8.36	15.55	10.15	5.41
	Dyn. I-RRIP	9.21	13.91	5.90	8.01
	SPP + Dyn. (unaware)	8.36	13.01	5.90	7.11
	SPP + Dyn. (aware)	8.37	12.99	5.88	7.12

Table 16: Load MPKI of the L1D, the L2C, and the L2C split into instruction and data loads.

	Load MPKI				Ratio to baseline			
Config	L1D	L2C	Instr.	Data	L1D	L2C	Instr.	Data
Baseline	9.21	16.34	10.11	6.23	1.000	1.000	1.000	1.000
SPP	8.36	15.55	10.15	5.41	0.876	0.901	1.009	0.823
Dyn. I-RRIP	9.21	13.91	5.90	8.01	1.000	0.900	0.631	1.242
SPP + Dyn. (unaware)	8.36	13.01	5.90	7.11	0.876	0.797	0.607	1.051
SPP + Dyn. (aware)	8.37	12.99	5.88	7.12	0.877	0.797	0.605	1.053

Table 17: Mean load MPKI (arithmetic), and each MPKI relative to the baseline (geometric mean of the per-trace ratios).

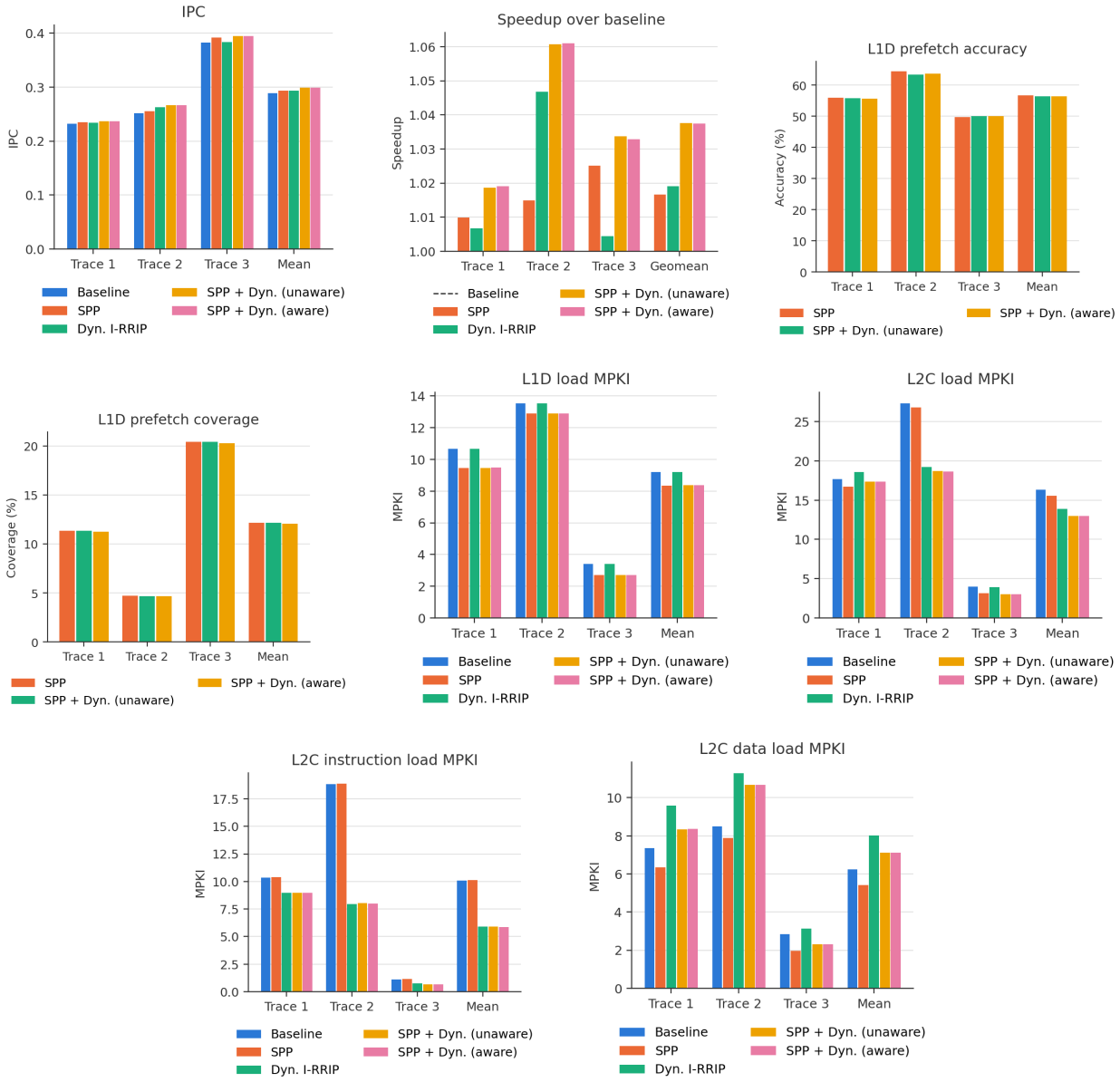


Figure 8: IPC, speedup over the baseline (dashed line), L1D prefetch accuracy and coverage, and load MPKI per trace.

SPP + Dynamic I-RRIP beats both on every trace. Mean speedup 1.0375, against 1.0167 for SPP and 1.0191 for Dynamic I-RRIP. Per trace it gains 1.90%, 6.11% and 3.29%; the best single component gains 0.99% (SPP), 4.68% (Dynamic I-RRIP) and 2.52% (SPP).

Why the gains add up. The two remove different misses. SPP removes L1D data misses (L1D load MPKI 0.88 of the baseline, with or without Dynamic I-RRIP, since the L2C policy does not change what the L1D holds). Dynamic I-RRIP removes L2C instruction misses (0.63 of the baseline alone, 0.60 combined); its extra L2C data misses (1.24 of the baseline alone) are mostly paid back by SPP (1.05 combined). The speedup is close to the product of the two ($1.0167 \times 1.0191 = 1.0361$).

What the rule adds. Little IPC. Accuracy and coverage are measured at the L1D and do not

change (56.4% and 12.1%). The mean speedup is 1.0375 with and without the rule; per trace the difference is +0.03%, +0.05% and −0.08%. Trace2, with the most L2C instruction misses, gets fewer of them (8.03 to 7.97 MPKI). On Trace3 the rule has 0.3% more L2C misses, although it lowers Trace3's misses in both pinned modes; both runs make the same 7 mode decisions, and we did not find the cause. The rule is small because the followers are mostly in I-RRIP mode, which already evicts prefetched lines first.